

ВВЕДЕНИЕ В ПРОГРАММИРОВАНИЕ.

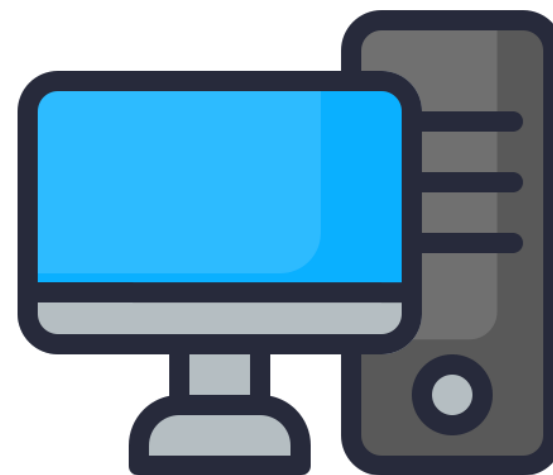


Как компьютеры «понимают» нас?

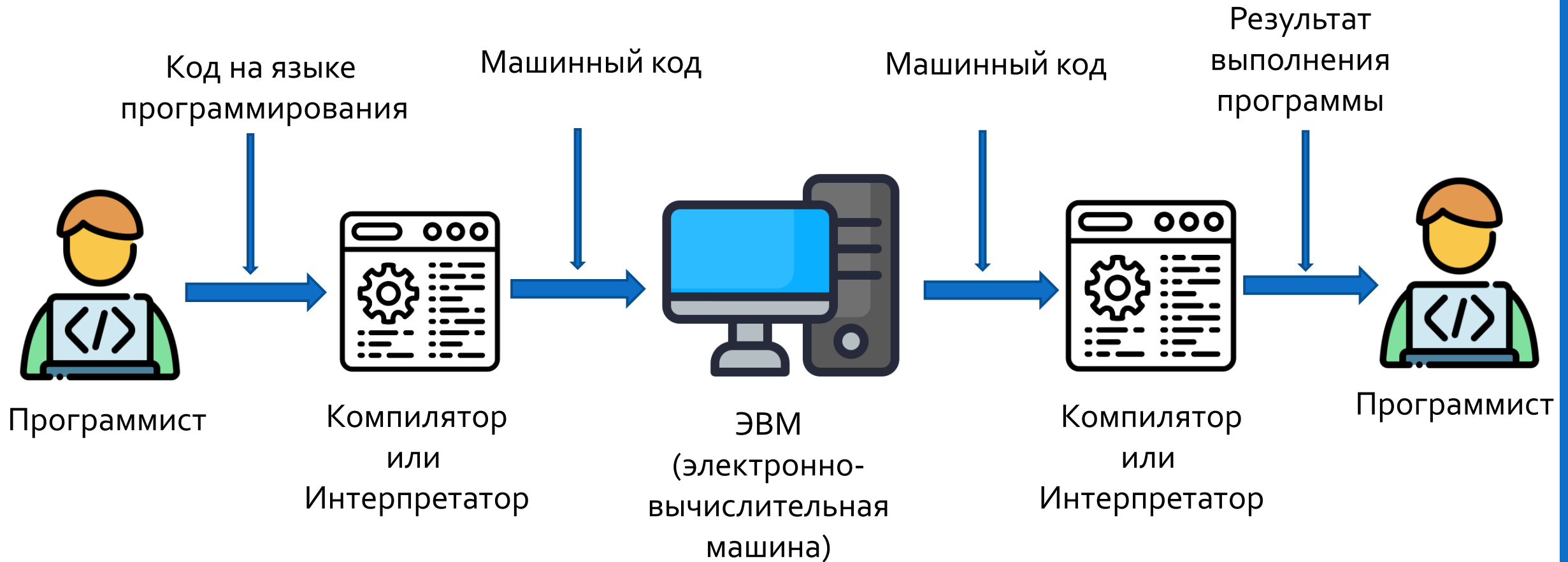
Компьютер как и любая другая **ЭВМ** (электронно-вычислительная машина) работает по заранее заданному набору **инструкций** — шагов, которые устройство выполняет в строгом порядке.

Например:

1. СОЗДАТЬ новый белый лист.
2. ВЗЯТЬ красный карандаш.
3. ОПУСТИТЬ карандаш в точку (10, 10).
4. ПРОВЕСТИ линию ВПРАВО на 100 шагов.
5. ПРОВЕСТИ линию ВНИЗ на 100 шагов.
6. ПРОВЕСТИ линию ВЛЕВО на 100 шагов.
7. ПРОВЕСТИ линию ВВЕРХ на 100 шагов.
8. ПОДНЯТЬ карандаш.
9. СООБЩИТЬ: "Квадрат готов!"



Как компьютеры «понимают» нас?



Машинный код – язык, понятный компьютеру

Компьютеры «разговаривают» на языке **машинного кода**. Это набор инструкций, закодированных в виде чисел, которые понимает процессор. Машинный код пишется в двоичном формате, состоящем из нулей и единиц.

Пример машинного кода:

10101100 00001111 11001010 00000001

Однако машинный код и двоичный код — это не одно и то же.

Двоичный код — это всего лишь система счисления, представляющая информацию (текст, числа, команды) в виде 0 и 1. А **машинный код** — это конкретные команды, которые выполняет процессор, записанные в двоичной форме.

Проблема в том, что писать и понимать такой код сложно. Поэтому придумали языки программирования.

Языки программирования – посредник между человеком и машиной

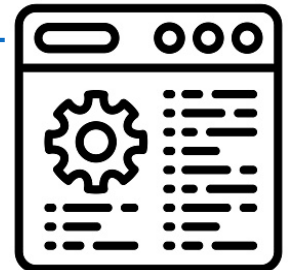
Языки программирования позволяют писать команды в привычной для человека форме, а затем специальные программы (компиляторы и интерпретаторы) переводят их в машинный код.

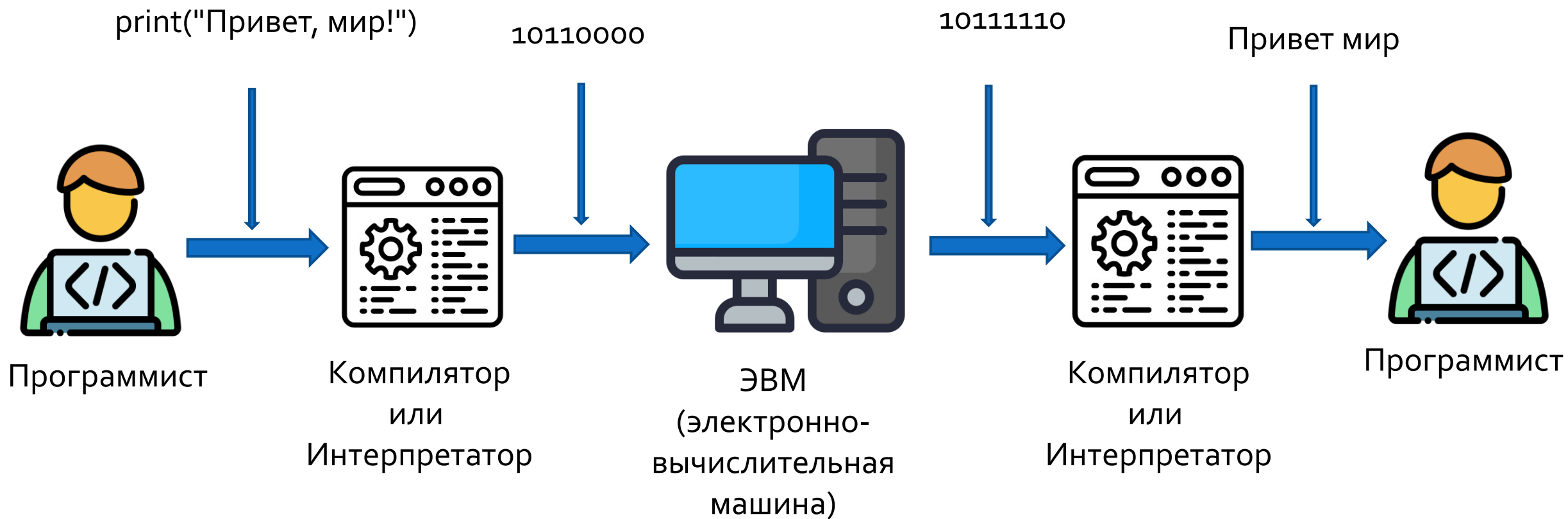
Например, вместо машинного кода мы можем написать на языке Python:

```
print("Привет, мир!")
```

Языки программирования можно сравнить с переводчиком:

- Мы говорим на человеческом языке (Python, Java, C++).
- Программа-переводчик (компилятор или интерпретатор) преобразует эти машинные команды.
- Компьютер выполняет эти команды и выдаёт результат.





Что такое программирование?

Программирование – это процесс написания инструкций на языке программирования, которые компьютер выполняет для решения определённой задачи.

Программист – это специалист, который создаёт программы с помощью языков программирования.

Программисты:

- Анализируют задачи и придумывают решения.
- Пишут код и тестируют его.
- Оптимизируют программы, делая их быстрее и удобнее.
- Работают с алгоритмами и структурами данных.

ЗНАКОМСТВО С PYTHON.



Python

Python – это мощный и универсальный язык, который широко используется в различных сферах IT. Его популярность обусловлена простотой синтаксиса, читаемостью кода и огромным сообществом разработчиков.

Python применяется в самых разных областях:

- **Веб-разработка** – создание сайтов и веб-приложений.
- **Анализ данных и машинное обучение** – обработка больших данных, нейросети, искусственный интеллект.
- **Автоматизация задач** – написание скриптов, которые выполняют рутинные процессы.
- **Кибербезопасность** – создание инструментов для тестирования безопасности.
- **Разработка игр** – создание игр на основе Pygame.
- **Встраиваемые системы и IoT** – программирование микроконтроллеров.

Интерпретируемый язык программирования

Python относится к **интерпретируемым языкам** программирования.

- В отличие от компилируемых языков (например, С или Java), Python не требует предварительной компиляции в машинный код.
- Вместо этого специальная программа – интерпретатор Python – выполняет код построчно, переводя его в машинные инструкции на лету.

Это можно сравнить с тем, как человек читает книгу.

- Если код компилируемый – это значит, что сначала вся книга переводится целиком, а затем мы можем её прочитать.
- Если код интерпретируемый (как в Python), то компьютер читает и выполняет код построчно, так же как человек читает книгу по одной строке за раз.

Подготовка среды разработки

Среда разработки — программа или сервис, которые переводят(интерпретируют или компилируют) код, который мы пишем.

Комментарии в Python

Комментарии — это участок в программе, который интерпретатор не считывает и не переводит. Для того чтобы написать комментарий необходимо перед текстом поставить **#**. Комментарий не обязательно должен начинаться с начала строки, их можно писать в любой момент строки, главное помнить, что **справа от знака #** всё считается комментарием и работать не будет.

```
1  # Комментарий 1
2  print("Hello world!") # Комментарий 2|
```

Ввод и вывод данных в Python

print — это команда, которая даёт компьютеру инструкцию: “Выведи что-то на экран”.

```
print("Hello World")  
#Программа выведет на экран: "Hello World"
```

Для вывода текста на новую строку можно написать команду **print** с новой строки или поставить символ переноса строки **\n** в месте начала новой строки.

*учтите, что пробел после **\n** отобразится на экране, если хотите тест в одну строку, то после **\n** надо сразу писать свой текст

Код:

```
print("Hello world!\n I'm Elvira")  
print("Nice to meet you!")  
print("Bye!")
```

Результат:

Your Output

```
Hello world!  
 I'm Elvira  
Nice to meet you!  
Bye!
```

Ввод и вывод данных в Python

`input()` — команда, которая даёт возможность пользователю(нам) ввести данные в консоль. Это может быть ответ на вопрос или сообщение, которое надо вывести или передать. Также как и команда `print()`, может выводить сообщения, если также как и с командой `print()`, указать текст внутри скобок.

Чтобы запомнить ответ пользователя, необходимо его записать в переменную.

Переменная – это способ сохранить какую-то информацию в программе, чтобы можно было её потом использовать, изменить или показать.

В создании переменной, придумывание имени является первым шагом для её создания.

Переменные в Python

Имя переменной можно придумать самому, но есть несколько правил:

- Начинается с буквы или подчёркивания `_`
- Нельзя начинать с цифры
- Нельзя использовать пробелы
- Нельзя использовать слова, которые уже заняты Python (например, `print`, `input` — команды с которыми мы работали и др.). Все занятые слова выполняют какое-то действие или команду, поэтому Python не поймёт хотим ли мы создать переменную или выполнить действие.
- Имена чувствительны к регистру (например, `name`, `Name`, `NAME` — три разные переменные)

Также хорошим тоном считается давать название переменной, отражающую суть того, что она хранит.



Примеры хороших имён:

`age`, `user_name`, `score`, `_secret`

Плохие имена (нельзя так):

`zname`, `user name`, `print`



Переменные в Python

После того, как придумали имя, создадим переменную. Запишем её название в программе, далее необходимо поставить знак «=», в программировании этот знак имеет значение «Присвоить», после этого знака мы пишем значение, которое хотим внести в нашу переменную (присваиваем переменной значение)

В Python переменная записывается так:

```
name = "Маша"
```

Здесь:

- **name** — имя переменной
- «=» — знак присваивания
- «Маша» — значение, которое мы сохраняем

Ввод и вывод данных в Python

Присваиваем переменной (с помощью знака **=**) значение результата выполнения команды **input()** , то есть сохраняем ответ пользователя в переменной.

Записываем в скобках и кавычках (весь текст пишем **обязательно** в них) вопрос который увидит пользователь.

Результат показываем на экране с помощью команды **print**.

Пишем вводный текст в кавычках и выводим ответ пользователя с помощью переменной

Обязательно пишем **+** если объединяем две строки(используем конкатенацию), например текст и переменную или две переменные, как в математическом примере $1+1=2$

Код:

```
name = input("Как тебя зовут? ")  
print("Привет, " + name + "!")
```

Результат:

Sample Input

Эльвира

Your Output

Как тебя зовут? Привет, Эльвира!

ПЕРЕМЕННЫЕ И ДАННЫЕ В PYTHON.



Типы данных в Python

Переменные в Python могут хранить разные **типы данных** — числа, слова, и даже правду и ложь!

Вернёмся к примеру, что переменная это коробка, но переменных теперь будет 4.

- В одну коробку кладём текст — это строка.
- В другую — яблоки поштучно — это целое число.
- В третью — молоко, которое наливаем не до конца — это дробное число.
- А в последнюю кладём табличку «Да» или «Нет» — это логика (True/False).

Чтобы не перепутать коробки, на них пишут, что внутри.

Вот Python делает так же — он «смотрит», что ты положил, и запоминает тип данных.

Основные типы данных в Python

- **Строка (str)** — это текст (от слова string - строка)

Это всё, что находится внутри кавычек: " , или ' '. Даже числа в кавычках считаются строкой!

```
1 name_1="Маша"  
2 name_2='Мария'  
3 age="12"
```

- **Числа**

int (от слова integer — целое число):

```
age=12
```

float (число с запятой, но в коде — с точкой):

```
height=155.5
```

- **Логический тип (bool)** — это правда или ложь

```
is_raining = True  
is_sunny = False
```

В Python есть два логических значения: **True (правда)** и **False (ложь)**. Они пишутся с заглавной буквы!

Основные типы данных в Python

Для работы с числовыми типами данных, в Python представлены все основные **математические операции**, для использования которых надо использовать следующие операторы:

- Оператор «+» — сложение
- Оператор «-» — вычитание
- Оператор «*» — умножение
- Оператор «/» — деление с остатком(результатом деления всегда будет дробное число (тип float), даже если всё делится без остатка)

например $2/4=0,5$ $2/2=1.00$

- Оператор «//» — деление без остатка(результатом деления всегда будет дробное число (тип float), даже если всё делится без остатка)

например $2/4=0$ $2/2=1$

Преобразование типов данных

- Однако бывают ситуации, когда уже существующую переменную надо перевести из одного типа в другой, чтобы программа работала правильно.

```
1 age = input("Сколько тебе лет?")
2 print(age + 5)
```

Ошибка:

```
Сколько тебе лет?23
```

```
Traceback (most recent call last):
```

```
File "c:\Users\Home\VS code Projects\prj1\main.py", line 2, in <module>
    print(age + 5)
```

```
TypeError: can only concatenate str (not "int") to str
```

```
PS C:\Users\Home\VS code Projects\prj1\
```

Преобразование типов данных

Правильный вариант с преобразованием

```
1 age = input("Сколько тебе лет?")
2 age = int(age)
3 print(age + 5)
```

Результат:

```
Сколько тебе лет?23
28
```

Такие команды «переводчики» есть у каждого типа данных : `int()`, `float()`, `str()` и т.д.

Преобразовать можно **не всё во всё**. Например:

Чтобы превратить строку в число (`int()` или `float()`), в строке **должны быть только цифры**.

Код:

```
name = "Вася"
print(int(name))
```

Ошибка:

```
Traceback (most recent call last):
  File "c:\Users\Home\VS code Projects\prj1\main.py", line 2, in <module>
    print(int(name))
          ~~~~~^~~~~~
ValueError: invalid literal for int() with base 10: 'Вася'
```

Преобразование типов данных

Преобразование в `bool()` зависит от того, пустая переменная или нет:

```
print(bool("Привет")) # True
print(bool(""))       # False
print(bool(0))         # False
print(bool(5))         # True
```


Спасибо за внимание!

